Final Report

Team 82

1 The Solution

PaisaPath is an offline-first progressive web app that teaches financial literacy to Grade 11–12 students by having them *live* a fictional character's money year rather than read rules about it. The problem statement, *Empowering the Next Generation Through Financial Literacy*, asks for a scalable way to cover five areas — banking and digital payments, budgeting, saving and borrowing, investing, and fraud safety — for a pilot of about 1,500 students in Hyderabad, delivered through NGO partners.

Our reading of the problem has not changed since M1: money lessons fail because consequences arrive weeks later and cost too much to learn from. A student told to "keep an emergency fund" has no way to feel what happens when they do not. So the app makes consequences immediate, cheap and reversible. The main mode is **Paisapur**, a top-down tile town in the style of a classic handheld RPG. The student walks the town, enters buildings, and plays a prologue plus six chapters — bank, budget, save, borrow, invest, protect — each closing with a "master's challenge" gate. The character can overspend, fall for a simulated scam or miss a goal; the student can then rewind to an earlier decision through the **What-If Machine**, choose differently and compare the two lives. Both timelines replay from the same seed, so any difference comes from the decision and not from luck. A rival character, re-simulated from that same seed, gives a constant side-by-side contrast: the student always sees what the other choice was worth.

Every activity runs the same seven-step loop — *story, decision, consequence, "why", concept, check, reward.* The consequence always lands before the explanation, so the "why" answers a question the student already has. Each check is tagged with a concept ID and, when wrong, a *misconception* ID. That is what later lets a facilitator see not merely that a class scored badly, but which wrong idea it holds.

The setting is deliberately hard: low-end and often shared phones, unreliable classroom internet, four languages, and facilitators who need to see where a class is stuck. Because the users are minors, the app never asks for or stores real money, family or message data — a class code and a picture PIN are the only identity — and it never names or recommends a real financial product. Both rules are enforced by tests rather than left to reviewer discipline.

2 Approach and Architecture

The app is a React 19 / TypeScript / Vite PWA. The architecture is organised around one rule: **the simulation is pure and the content is data.**

- `core/` — a pure, deterministic TypeScript simulation: the money engine, a seeded fictional market, world/collision maps, a story-script interpreter, rival simulation, and the timeline (checkpoint, rewind, compare). It imports no React, no database and no network. Clocks and randomness are injected rather than called, so the same seed and the same choices always produce the same state. This is what makes rewind and compare trustworthy rather than approximate — without it a "what if" could differ because of luck, and the whole lesson would be noise.
- `platform/` — the only code allowed to touch IndexedDB, the network or the backend: saves, snapshots, profiles, the telemetry outbox, sync status and translation. Everything else imports from a single barrel, so storage or transport can change without touching gameplay.
- `content/` — 395 JSON files: story scripts, world maps, lessons, glossaries, quizzes, scenarios, the concept registry and the four language bundles. New scenarios and lessons need no code.
- `ui/` — a presentational pixel/tile design kit shared by every screen.

These boundaries are not conventions but **lint-enforced** import rules, so a pull request that reaches into the database from game logic fails CI. A standalone content validator (zero dependencies, runs from the repository root) independently checks IDs, effect paths, story steps, warps, NPC scripts and translation key coverage, and gates CI alongside types, lint, tests and the production build.

Content as data. Story scripts are JSON step lists (`say`, `choice`, `if`, `do`, `quiz`, `concept`, `minigame`, ...) with conditions written as strings such as `chapter>=2 && !flag:x`. A `minigame` step can stage a scene, convert the game's result back into story money, and run follow-up dialogue for a choice made *inside* the game. That

is how the mini-games stay in one economy rather than being a separate arcade bolted on: a game reads the character's real income, bills and prices, and what the student does there changes the story's balance.

Telemetry and privacy. A gameplay event carries only coded IDs — a pseudonymous player ID made on the device, a class join code, and activity, concept, choice and misconception IDs. There are no names, no free text and no device identifiers, and the format is shared between the app and the ingest function so the two cannot drift. Event IDs are ULIDs, which makes uploads idempotent: a retry after a dropped connection cannot double-count. Privacy continues into the dashboard — a concept bar stays hidden until at least 3 students have answered it, so a small class cannot be used to single anyone out.

Offline-first sync. Play never waits on a network. Events queue in an IndexedDB outbox and upload in batches of at most 200 with exponential back-off; class unlocks return in the same response. Language packs are split out of the main bundle and fetched on demand, so a Telugu profile does not pay to download Hindi and Urdu at boot.

Translation pipeline. Every student-visible string goes through `t('namespace.key')`, with per-key fallback to English, plurals via `Intl.PluralRules`, and rupee formatting pinned to Indian digit grouping in all four languages. Two authoring scripts export every English string to a tab-separated sheet a translator can open in any spreadsheet, and import it back with placeholder guards, so translating the app does not require a developer.

3 Features

- **Paisapur (main mode).** Prologue and six chapters, 53 story scripts across 10 files, a walkable town with shops, a bank, a lender and NPCs, a seeded market, a needs/wants/goal economy, badges, and the What-If Machine with its rival timeline.
- **Mini-games.** 15 Everyday Money games (document hunt, bank counter, jar split, goal builder, spending detective, surprise month, debt ladder, . . .), 9 investing modules and 11 Stay Safe simulators — 52 registered IDs in total. Games read money only through shared context helpers, and a coherence test fails the build if a game and the story disagree about income, bills or prices.
- **Investing.** Task-based rooms with typed answers, hints and points; interactive labs; a Time Machine that runs 500 simulated futures and reports a *range* with its assumptions, never a single number; a 12-week stock-market simulator with six fictional companies and a coach that never says what to buy; a mutual-fund challenge; and an investing/gambling/scam sorter.
- **Stay Safe.** A simulated phone with SMS, chat, email, calls and search; a Scam Lens that annotates the tells in a message; a UPI simulator — pay by QR, then receive a fake collect request, the trap being that one looks like the other; Scam Duels; XP, stars, ranks and badges.
- **Facilitator dashboard.** Create classes, share a join code, unlock stages, and read an Insights tab that ranks the concepts a class finds hardest, lists the most common misconceptions, and flags students below 50% accuracy once they have answered at least 6 questions — so support is suggested on evidence rather than on a hunch. Row-level security means a facilitator sees only their own classes. Students never log in to the backend at all.
- **Offline and sync.** Installable PWA; saves and snapshots in IndexedDB; a telemetry outbox that batches, backs off and never sends the same event twice; unlocks flowing back down; and a visible sync status, so a facilitator knows whether what they are reading is current.
- **Languages and accessibility.** Four languages (English, Telugu, Hindi, Urdu) with right-to-left layout for Urdu, a "Listen" button on text, and a `timersOff` setting that keeps every timer visible but stops it expiring, so a slower reader is never cut off. Colour choices are held to WCAG AA by test.
- **Authoring tooling.** A content validator, a string exporter and importer, and a tile-index dev page, so new scenarios, lessons and languages can be added by people who do not write code.

4 Results

All figures below are measured on the merged `main` branch, not carried over from the per-branch numbers reported in M2.

Check	Result	What it covers
Test suite (Vitest)	**1703 / 1703** pass, 85 files	money engine, market, story content, rewind, duels, all mini-games, sync outbox, facilitator insights, translation
Types (`tsc -b`)	clean	strict mode across 452 source files (≈37.9k lines)
Lint (oxlint)	0 errors	includes the folder-boundary import rules
Content validator	**PASS, 0 errors**	137 concepts, 81 misconceptions, 93 scenario files, 53 story scripts, 52 game IDs, 6799 English keys, 3812 referenced keys
Brand-name guardrail	502 checks pass	no real bank, fund, app or company name appears in any lesson, game or scenario text
Investing disclaimers	12 checks pass	every investing route shows the "simulation, not advice" banner; no screen shows a lone projected number
Colour contrast	23 checks pass	every text/background token pair $\geq$ 4.5:1 (WCAG AA)

Determinism, tested end to end. The same seed produces the same market and the same story outcome. A bot plays the entire year through all six chapters as part of the suite, so a content change that makes the story unwinnable fails CI rather than reaching a classroom. Rewinding leaves the original timeline untouched, which is what makes the side-by-side comparison honest. Every number a lesson room quotes is checked against the shared assumptions file, so the prose and the maths cannot drift apart.

The guardrails are the headline result. The brief rules out naming real products to minors, so we made that mechanical instead of editorial: a word list of real bank, fund, app and company names is scanned across every lesson, game and scenario string, and the build fails on a single hit (502 checks). Alongside it, 12 checks assert that no investing screen can render a lone projected number without its range and assumptions. Neither can regress silently.

Fairness of the fund model. The mutual-fund engine is tested so that active and sector funds grow about the same as the index fund after cost, and no text calls any fund best — the lesson is about cost and diversification, not about picking a winner.

Translation coverage. The Telugu, Hindi and Urdu bundles are populated and verified against English key by key by the validator, which treats a key present in a translation but absent from English as a hard error.

Language	Keys translated	Coverage	Complete namespaces
Hindi	5716 / 6799	84.1%	all of Paisapur, Stay Safe, Scam Duels, arcade, setup
Urdu	5236 / 6799	77.0%	as above, right-to-left verified
Telugu	4770 / 6799	70.2%	as above

57 of the 82 string namespaces are complete in all four languages, and **Paisapur itself — the main mode — is 100% complete in all four** (1456 keys each, 4368 translations); 15,722 of 20,397 non-English strings are translated overall. The remainder is standalone lesson bundles, none of which sit on the Paisapur story path, and a missing key falls back to English rather than breaking the screen.

Integration. The three parallel branches reported separately in M2 are merged into a single app. The investing rooms and Stay Safe simulators are reachable from inside the town and cross-linked by concept, so a student who meets `loan_app_trap` in the story is pointed at the simulator that drills it, and every mode logs its events to the same dashboard.

Coverage of the five areas in the brief. Each area is taught twice: once as a chapter of the story, where it has consequences, and again as focused practice that can be replayed on its own.

Area in the brief	In the story	Focused practice
Banking and digital payments	Chapter 1 — opening an account	document hunt, bank counter (account type, nominee, PIN), UPI simulator
Budgeting	Chapter 2 — the first month	jar split, needs/wants and budget lessons, trade-offs
Saving and borrowing	Chapters 3–4 — a goal, then a shock	goal builder, spending detective, surprise month, lender comparison, debt ladder
Investing	Chapter 5 — growing it	task rooms and labs, stock-market simulator, mutual-fund challenge, Time Machine
Staying safe from fraud	Chapter 6 — protecting it	phone simulator, Scam Lens, Scam Duels, investing/gambling/scam sorter